

## AIGC检测 · 简洁报告单

NO:CNKIAIGC2026SG\_20260521123513345

检测时间: 2026-05-21 12:35:13

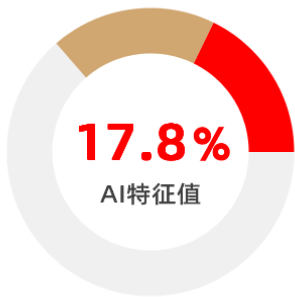
篇名: 面向深度学习系统模糊测试的测试用例智能管理与分析

作者: 郭俊杰

单位:

文件名: 面向深度学习系统模糊测试的测试用例智能管理与分析.docx

全文检测结果 知网AIGC检测 <https://cx.cnki.net>



AI特征值: 17.8%

AI特征字符数: 6082

总字符数: 34170

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

### AIGC片段分布图

前部20%

AI特征值: 16.1%

AI特征字符数: 1101

中部60%

AI特征值: 18.7%

AI特征字符数: 3840

后部20%

AI特征值: 16.7%

AI特征字符数: 1141



0

34170

■ AI特征显著 ■ AI特征疑似 ■ 未标识部分

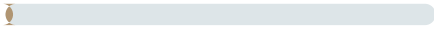
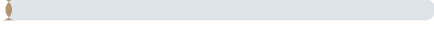
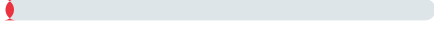
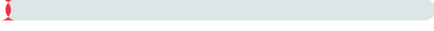
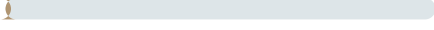
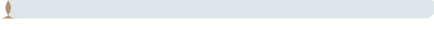
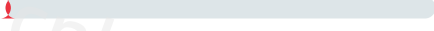
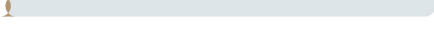
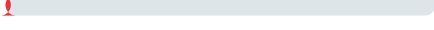
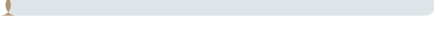
### 分段检测结果

| 序号 | AI特征值 | AI特征字符数/章节字数 | 章节(部分)名称                      |
|----|-------|--------------|-------------------------------|
| 1  | 22.5% | 2705 / 12028 | 面向深度学习系统模糊测试的测试用例智能管理与分析_第1部分 |
| 2  | 19.5% | 2349 / 12044 | 面向深度学习系统模糊测试的测试用例智能管理与分析_第2部分 |
| 3  | 15.0% | 1804 / 12042 | 面向深度学习系统模糊测试的测试用例智能管理与分析_第3部分 |

1.面向深度学习系统模糊测试的测试用例智能管理与分析\_第1部分

AI特征值：22.5% AI特征字符数/章节(部分)字符数：2705/12028

片段指标列表

| 序号 | 片段名称 | 字符数 | AI特征 |                                                                                      |      |
|----|------|-----|------|--------------------------------------------------------------------------------------|------|
| 1  | 片段1  | 207 | 疑似   |    | 1.7% |
| 2  | 片段2  | 123 | 疑似   |    | 1.0% |
| 3  | 片段3  | 176 | 疑似   |    | 1.5% |
| 4  | 片段4  | 172 | 显著   |    | 1.5% |
| 5  | 片段5  | 120 | 疑似   |    | 1.0% |
| 6  | 片段6  | 120 | 显著   |    | 1.0% |
| 7  | 片段7  | 288 | 疑似   |   | 2.4% |
| 8  | 片段8  | 137 | 显著   |  | 1.2% |
| 9  | 片段9  | 179 | 疑似   |  | 1.5% |
| 10 | 片段10 | 233 | 显著   |  | 2.0% |
| 11 | 片段11 | 140 | 显著   |  | 1.2% |
| 12 | 片段12 | 151 | 疑似   |  | 1.3% |
| 13 | 片段13 | 123 | 疑似   |  | 1.0% |
| 14 | 片段14 | 140 | 显著   |  | 1.2% |
| 15 | 片段15 | 137 | 显著   |  | 1.2% |
| 16 | 片段16 | 122 | 疑似   |  | 1.0% |
| 17 | 片段17 | 126 | 疑似   |  | 1.1% |
| 18 | 片段18 | 120 | 显著   |  | 1.0% |
| 19 | 片段19 | 142 | 疑似   |  | 1.2% |

|    |      |     |    |                        |      |
|----|------|-----|----|------------------------|------|
| 20 | 片段20 | 121 | 显著 | <div><div></div></div> | 1.0% |
| 21 | 片段21 | 129 | 显著 | <div><div></div></div> | 1.1% |
| 22 | 片段22 | 123 | 显著 | <div><div></div></div> | 1.0% |
| 23 | 片段23 | 128 | 疑似 | <div><div></div></div> | 1.1% |
| 24 | 片段24 | 120 | 显著 | <div><div></div></div> | 1.0% |
| 25 | 片段25 | 145 | 显著 | <div><div></div></div> | 1.2% |
| 26 | 片段26 | 134 | 显著 | <div><div></div></div> | 1.1% |
| 27 | 片段27 | 134 | 疑似 | <div><div></div></div> | 1.1% |
| 28 | 片段28 | 133 | 显著 | <div><div></div></div> | 1.1% |
| 29 | 片段29 | 134 | 疑似 | <div><div></div></div> | 1.1% |
| 30 | 片段30 | 162 | 显著 | <div><div></div></div> | 1.4% |
| 31 | 片段31 | 159 | 显著 | <div><div></div></div> | 1.3% |
| 32 | 片段32 | 131 | 显著 | <div><div></div></div> | 1.1% |
| 33 | 片段33 | 136 | 疑似 | <div><div></div></div> | 1.1% |
| 34 | 片段34 | 179 | 疑似 | <div><div></div></div> | 1.5% |
| 35 | 片段35 | 149 | 显著 | <div><div></div></div> | 1.3% |
| 36 | 片段36 | 127 | 疑似 | <div><div></div></div> | 1.1% |
| 37 | 片段37 | 172 | 疑似 | <div><div></div></div> | 1.5% |

片段详情

|       |     |          |          |                        |      |
|-------|-----|----------|----------|------------------------|------|
| NO. 1 | 片段1 | 字符数： 207 | AI特征： 疑似 | <div><div></div></div> | 1.7% |
|-------|-----|----------|----------|------------------------|------|

针对上述问题，论文设计并实现了一套结合语义检索和大语言模型（LLM）的模糊测试数据管理与分析系统。论文主要工作如下：

1. 针对模糊测试产物类型较多、统一管理不便的问题，设计并实现了测试数据管理功能。系统对测试脚本结构、执行状态、覆盖贡献数据、崩溃日志和语义向量进行统一关联，支持测试用例导入、列表查询、条件筛选、详情查看和相似用例召回，并通过结构化元数据和语义向量的协同存储维护测试用例之间的关联关系，

NO. 2      片段2      字符数： 123      AI特征： 疑似            1.0%

系统还在前端展示覆盖缺口、测试用例多样性分布和用例效能排行，并提供低效用例筛选和候选补盲测试脚本生成功能，用于辅助用户了解测试集覆盖情况和用例价值分布。3. 针对崩溃用例中故障线索和候选边界信息不易整合分析的问题，设计并实现了故障分析功能。

NO. 3      片段3      字符数： 176      AI特征： 疑似            1.5%

This thesis studies the management and analysis of fuzzing outputs for the PyTorch deep learning framework. Fuzz testing can generate many test cases with boundary parameters,

NO. 4      片段4      字符数： 172      AI特征： 显著            1.5%

operator combinations, and crash logs. However, these data are still difficult to use directly in later analysis, because fuzzing outputs are not easy to manage uniformly,

NO. 5      片段5      字符数： 120      AI特征： 疑似            1.0%

combined with semantic retrieval and large language models (LLMs). The main work is as follows: 1. For data management,

NO. 6      片段6      字符数： 120      AI特征： 显著            1.0%

a test data management function is designed and implemented to address the problem that fuzzing outputs have many types

|                                                                                                                                                                                                                                                                                                        |      |          |          |                        |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------|----------|------------------------|------|
| NO. 7                                                                                                                                                                                                                                                                                                  | 片段7  | 字符数: 288 | AI特征: 疑似 | <div><div></div></div> | 2.4% |
| <p>It supports test case import, list query, conditional filtering, detail view, and similar-case retrieval, and maintains relations between test cases through the coordinated storage of structured metadata and semantic vectors, providing a basis for later data analysis and fault analysis.</p> |      |          |          |                        |      |
| NO. 8                                                                                                                                                                                                                                                                                                  | 片段8  | 字符数: 137 | AI特征: 显著 | <div><div></div></div> | 1.2% |
| <p>and directions for additional testing. Based on the Coverage Efficiency Index (CEI) model, it considers the branch coverage contribution</p>                                                                                                                                                        |      |          |          |                        |      |
| NO. 9                                                                                                                                                                                                                                                                                                  | 片段9  | 字符数: 179 | AI特征: 疑似 | <div><div></div></div> | 1.5% |
| <p>and weighted computational cost of each test case, and filters low-benefit cases with a dynamic percentile threshold. The system also presents coverage gaps, test case diversity,</p>                                                                                                              |      |          |          |                        |      |
| NO. 10                                                                                                                                                                                                                                                                                                 | 片段10 | 字符数: 233 | AI特征: 显著 | <div><div></div></div> | 2.0% |
| <p>and case efficiency rankings on the front end, and provides low-efficiency case filtering and candidate gap-filling test script generation functions, helping users understand coverage status and the value distribution of test cases.</p>                                                        |      |          |          |                        |      |
| NO. 11                                                                                                                                                                                                                                                                                                 | 片段11 | 字符数: 140 | AI特征: 显著 | <div><div></div></div> | 1.2% |
| <p>the system provides fault localization assistance, similar-case display, candidate boundary-condition hints, and boundary comparison views.</p>                                                                                                                                                     |      |          |          |                        |      |
| NO. 12                                                                                                                                                                                                                                                                                                 | 片段12 | 字符数: 151 | AI特征: 疑似 | <div><div></div></div> | 1.3% |
| <p>and generates auxiliary diagnostic reports and candidate gap-filling test scripts. These functions provide references for users to review crash causes</p>                                                                                                                                          |      |          |          |                        |      |

NO. 13

片段13

字符数： 123

AI特征： 疑似



1.0%

对自动驾驶感知、医疗影像分析等对结果稳定性要求较高的场景，这种影响还可能进一步引起漏检、误判等问题。除数值问题外，底层内存越界、缓冲区溢出等也会影响框架服务的稳定性。包含异常张量形状或边界参数组合的计算图（Computation Graph）

NO. 14

片段14

字符数： 140

AI特征： 显著



1.2%

因此学术界和工业界逐渐把模糊测试（Fuzzing）引入到DL框架测试场景中。模糊测试用自动化算法生成包含畸形参数、极端边界值或特殊算子组合的输入，持续送入待测框架执行，以便发现普通测试样例不容易触发的问题。在DL框架场景里，模糊测试生成的输入一般不是简单的字符串或标量参数，

NO. 15

片段15

字符数： 137

AI特征： 显著



1.2%

但格式差异较大，又分散在脚本文件、日志文本和覆盖率报告等不同位置。当测试任务长时间运行后，这些测试产物会很快积累起来。如果没有统一的数据组织方式，后续的用例筛选和故障分析都需要较多人工参与。具体来说，测试脚本需要和它对应的覆盖率记录、执行结果以及崩溃日志建立对应关系；

NO. 16

片段16

字符数： 122

AI特征： 疑似



1.0%

同一底层缺陷可能因为输入脚本、变量名或调用路径不同而产生不同的表层日志；反过来，日志中相似的错误词也可能对应不同算子、不同参数组合或不同后端的问题。传统的文本检索工具主要按字面特征匹配，不太能同时利用算子拓扑结构和历史相似用例等上下文信息。

NO. 17

片段17

字符数： 126

AI特征： 疑似



1.1%

在模糊测试过程中，有些用例只带来很少的新覆盖，却需要较长的执行时间或较多显存；也有一些用例没有直接触发崩溃，但可能覆盖了之前较少出现的算子组合。如果系统只根据“代码覆盖率”或“是否异常”来判断，就不容易区分哪些用例值得保留，哪些只是在重复消耗资源。

NO. 18

片段18

字符数： 120

AI特征： 显著



1.0%

后续设计主要围绕相似用例检索、低收益用例筛选和辅助诊断这三个问题展开。其中，覆盖效能指数（Coverage Efficiency Index, CEI）用于评价用例的覆盖收益，基于相似样本对比的Daikon约束分析用于整理候选边界条件，

|                                                                                                                                                                    |      |          |          |                        |       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------|----------|------------------------|-------|
| NO. 19                                                                                                                                                             | 片段19 | 字符数： 142 | AI特征： 疑似 | <div><div></div></div> | 1. 2% |
| <p>检索增强生成（Retrieval-Augmented Generation, RAG）用于生成辅助诊断报告。国内外研究现状</p> <p>深度学习框架模糊测试方法研究</p> <p>面向DL框架的测试研究中，早期工作多数集中在输入生成问题上。Pei等人提出的DeepXplore用神经元覆盖率描述模型内部状态，</p> |      |          |          |                        |       |
| NO. 20                                                                                                                                                             | 片段20 | 字符数： 121 | AI特征： 显著 | <div><div></div></div> | 1. 0% |
| <p>Deng等人进一步尝试用大语言模型（Large Language Model, LLM）</p> <p>生成框架测试用例，</p> <p>说明LLM能够学习部分框架API签名和参数关系，</p> <p>也能生成一部分可执行的测试代码。这些研究对DL框架测试输入的自动生成有直接帮助，</p>               |      |          |          |                        |       |
| NO. 21                                                                                                                                                             | 片段21 | 字符数： 129 | AI特征： 显著 | <div><div></div></div> | 1. 1% |
| <p>并基于历史测试结果完成用例筛选、故障辅助定位和候选补盲测试脚本生成。测试数据管理与效能评估相关研究</p> <p>测试用例管理领域已经积累了不少关于优先级排序和用例约简的研究。Rothermel等人提出了基于覆盖率的贪心排序策略，</p> <p>优先执行能够带来更多新代码分支的用例，</p>              |      |          |          |                        |       |
| NO. 22                                                                                                                                                             | 片段22 | 字符数： 123 | AI特征： 显著 | <div><div></div></div> | 1. 0% |
| <p>近几年向量数据库（Vector Database）技术的发展</p> <p>为非结构化测试产物管理提供了新的工具。以Milvus为代表的向量检索系统可以对稠密浮点向量</p> <p>执行近似最近邻（Approximate Nearest Neighbor, ANN）查询，</p>                    |      |          |          |                        |       |
| NO. 23                                                                                                                                                             | 片段23 | 字符数： 128 | AI特征： 疑似 | <div><div></div></div> | 1. 1% |
| <p>不过，向量检索本身并不能替代覆盖率分析、用例成本评估或候选边界条件判断。它更适合回答“哪些历史样本和当前样本相似”，</p> <p>不能直接说明某个样本是否值得继续执行，</p> <p>也不能单独判断崩溃是不是由某个参数条件触发。因此本文采用MySQL和Milvus协同的存储方式，</p>               |      |          |          |                        |       |
| NO. 24                                                                                                                                                             | 片段24 | 字符数： 120 | AI特征： 显著 | <div><div></div></div> | 1. 0% |
| <p>在缺陷修复方面，也有研究尝试让LLM结合错误上下文生成候选补丁。这些工作说明LLM可以参与测试脚本生成、错误解释和候选修复建议等环节。放到DL框架测试场景中，</p> <p>LLM更适合处理API调用模式、参数组合和测试脚本模板这类与代码语义相关的问题。</p>                             |      |          |          |                        |       |

NO. 25 片段25 字符数: 145 AI特征: 显著  1.2%

检索增强生成 (Retrieval-Augmented Generation, RAG) 架构  
在模型推理之前先召回外部知识库里的相关上下文，  
可以在一定程度上约束模型输出。和直接调用LLM相比，  
RAG更适合把项目内的测试上下文提供给模型。现有研究更多关注LLM生成测试用例或生成修复建议，

NO. 26 片段26 字符数: 134 AI特征: 显著  1.1%

系统需要完成日志解析、字段抽取、向量生成、索引构建和数据同步等流程，  
让原本分散在文件系统和日志目录里的测试产物可以被统一查询和分析。第二项是测试集多样性观察和效能量化。  
系统用降维映射算法把高维测试用例特征投影到低维平面，  
辅助观察测试集在语义空间中的聚集和空缺情况。

NO. 27 片段27 字符数: 134 AI特征: 疑似  1.1%

同时，系统构建了覆盖效能指数 (Coverage Efficiency Index, CEI) 评估模型，  
把单个用例的覆盖贡献和执行成本结合起来，  
用于识别覆盖收益较低但执行开销较高的冗余用例。和只统计覆盖率不同，CEI模型关注的是用例在覆盖收益和运行代价之间的关系，

NO. 28 片段28 字符数: 133 AI特征: 显著  1.1%

在此基础上，系统把相似用例上下文、可疑约束和复现相关信息写入RAG提示词模板，  
由LLM生成辅助诊断报告和候选补盲测试脚本。围绕上述目标，本文先实现了测试脚本和崩溃日志的解析流程，  
把日志文本和脚本结构信息转换为语义向量。接下来，系统用MySQL保存结构化元数据，

NO. 29 片段29 字符数: 134 AI特征: 疑似  1.1%

本文还在插桩编译过的PyTorch环境下完成了千例规模的物理对照实验，  
对CEI筛选、边界条件分析和系统性能做验证。相关技术与理论基础  
深度学习框架与模糊测试技术  
深度学习框架的体系结构  
目前常见的DL框架大多采用分层结构。以PyTorch为例，自上而下可以分为三层。

NO. 30 片段30 字符数: 162 AI特征: 显著  1.4%



最上层是面向用户的Python接口，对外提供张量操作、自动微分和神经网络模块。中间层把用户写的运算关系组织起来，  
执行时会形成计算图或中间表示（Intermediate Representation）。为了便于后续分析，本文把算子之间的数据依赖看作一个有向无环图（Directed Acyclic Graph, DAG），

NO. 31

片段31

字符数：159

AI特征：显著

1.3%

把这些输入送给待测程序执行，再观察程序运行时是否出现异常信号。按输入的构造方式，Fuzzing一般可以分为两类：  
基于变异（Mutation-based）的方法从已有的种子输入出发，  
通过对字节做翻转、插入或删除来产生新的测试用例；  
基于生成（Generation-based）的方法则按照输入格式的语法或语义模型，

NO. 32

片段32

字符数：131

AI特征：显著

1.1%

传统的词袋模型或TF-IDF方法主要根据词项出现的频率得到稀疏向量，  
可以反映文本里出现了哪些关键词，  
但对词序、上下文和同义表达的处理能力比较有限。这一点在DL框架崩溃日志上比较明显：  
同一类底层异常可能因为变量名或调用路径不同，  
在文本上呈现出较大的差别；

NO. 33

片段33

字符数：136

AI特征：疑似

1.1%

反过来，两条用词相近的日志也不一定来自同一种算子或后端问题。因此本文系统不能只靠关键词匹配，  
还需要一种能够反映句子语义接近程度的编码方法。Reimers等人提出的Sentence-BERT  
在预训练语言模型的基础上引入了孪生网络（Siamese Network）结构，

NO. 34

片段34

字符数：179

AI特征：疑似

1.5%

再把得到的向量标识和MySQL中的结构化元数据关联起来保存。这些向量后续用于相似用例的检索和测试集多样性分析。高维向量近似最近邻检索  
当向量的数量不断增多之后，  
如果每次查询都和库里所有向量逐一计算距离，  
在线检索的耗时会明显上升。近似最近邻（Approximate Nearest Neighbor, ANN）检索一般会用少量精度损失来换取较快的查询速度。

NO. 35

片段35

字符数：149

AI特征：显著

1.3%

本文系统需要在已有的脚本和日志向量中较快地找到相似的样本，所以需要一个专门的向量检索系统。Milvus是一个面向向量数据的开源检索系统，支持多种索引结构和距离度量。本课题选用IVF\_FLAT（Inverted File Index with Flat quantization）作为主索引结构。

NO. 36

片段36

字符数：127

AI特征：疑似

1.1%

这个索引会先通过聚类把向量空间划分为若干Voronoi单元，查询时只在离查询点最近的几个单元里继续比较，把搜索范围从全部样本缩小到局部。这种方式和本文系统先召回候选相似用例、再结合结构化字段补全信息的查询过程相匹配。向量之间的距离度量使用余弦相似度。

NO. 37

片段37

字符数：172

AI特征：疑似

1.5%

所以在相似崩溃召回、相似脚本检索以及RAG上下文构造前的候选筛选里，系统都使用这个度量。降维分析与不变量推断方法  
主成分分析原理  
本文系统为每条测试用例生成的语义向量是384维。这类向量便于机器检索和相似度计算，但研究人员不容易直接从高维坐标看出测试集的分布。主成分分析（Principal Component Analysis, PCA）

2.面向深度学习系统模糊测试的测试用例智能管理与分析\_第2部分

AI特征值：19.5%      AI特征字符数/章节(部分)字符数：2349/12044

片段指标列表

| 序号  | 片段名称 | 字符数 | AI特征 |             |      |
|-----|------|-----|------|-------------|------|
| 101 | 片段1  | 159 | 疑似   | <div></div> | 1.3% |
| 102 | 片段2  | 124 | 疑似   | <div></div> | 1.0% |
| 103 | 片段3  | 126 | 疑似   | <div></div> | 1.1% |
| 104 | 片段4  | 132 | 显著   | <div></div> | 1.1% |
| 105 | 片段5  | 137 | 显著   | <div></div> | 1.2% |
| 106 | 片段6  | 147 | 疑似   | <div></div> | 1.2% |
| 107 | 片段7  | 125 | 疑似   | <div></div> | 1.1% |

|     |      |     |    |                                                                                     |      |
|-----|------|-----|----|-------------------------------------------------------------------------------------|------|
| 108 | 片段8  | 135 | 显著 |      | 1.1% |
| 109 | 片段9  | 125 | 疑似 |    | 1.1% |
| 110 | 片段10 | 123 | 显著 |    | 1.0% |
| 111 | 片段11 | 122 | 疑似 |    | 1.0% |
| 112 | 片段12 | 125 | 疑似 |    | 1.1% |
| 113 | 片段13 | 140 | 显著 |    | 1.2% |
| 114 | 片段14 | 120 | 显著 |    | 1.0% |
| 115 | 片段15 | 132 | 疑似 |    | 1.1% |
| 116 | 片段16 | 143 | 显著 |    | 1.2% |
| 117 | 片段17 | 133 | 显著 |    | 1.1% |
| 118 | 片段18 | 197 | 显著 |  | 1.7% |
| 119 | 片段19 | 136 | 显著 |  | 1.2% |
| 120 | 片段20 | 141 | 显著 |  | 1.2% |
| 121 | 片段21 | 122 | 疑似 |  | 1.0% |
| 122 | 片段22 | 150 | 疑似 |  | 1.3% |
| 123 | 片段23 | 165 | 疑似 |  | 1.4% |
| 124 | 片段24 | 136 | 疑似 |  | 1.2% |
| 125 | 片段25 | 130 | 显著 |  | 1.1% |
| 126 | 片段26 | 145 | 疑似 |  | 1.2% |
| 127 | 片段27 | 124 | 显著 |  | 1.0% |
| 128 | 片段28 | 129 | 显著 |  | 1.1% |
| 129 | 片段29 | 137 | 显著 |  | 1.2% |

|     |      |     |    |                        |      |
|-----|------|-----|----|------------------------|------|
| 130 | 片段30 | 139 | 疑似 | <div><div></div></div> | 1.2% |
| 131 | 片段31 | 177 | 疑似 | <div><div></div></div> | 1.5% |
| 132 | 片段32 | 121 | 显著 | <div><div></div></div> | 1.0% |
| 133 | 片段33 | 131 | 疑似 | <div><div></div></div> | 1.1% |
| 134 | 片段34 | 171 | 显著 | <div><div></div></div> | 1.4% |
| 135 | 片段35 | 155 | 疑似 | <div><div></div></div> | 1.3% |
| 136 | 片段36 | 136 | 疑似 | <div><div></div></div> | 1.2% |
| 137 | 片段37 | 175 | 疑似 | <div><div></div></div> | 1.5% |

片段详情

|         |     |         |         |                        |      |                                                                                                                                                                                                                                                                                                                                                                                                |
|---------|-----|---------|---------|------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NO. 101 | 片段1 | 字符数：159 | AI特征：疑似 | <div><div></div></div> | 1.3% | <p>给定<math>n</math>个<math>d</math>维样本构成的数据矩阵<math>X \in \mathbb{R}^{n \times d}</math>（已中心化），PCA先计算样本协方差矩阵：</p> <p>其中<math>C \in \mathbb{R}^{d \times d}</math>是一个对称半正定矩阵。之后再对<math>C</math>做特征值分解：</p> <p>其中<math>\lambda_k</math>是第<math>k</math>个特征值，<math>v_k</math>是对应的单位特征向量。把特征值按从大到小排好后，取前<math>p</math>个特征向量组成投影矩阵<math>W = [v_1, \dots, v_p] \in \mathbb{R}^{d \times p}</math>。</p> |
| NO. 102 | 片段2 | 字符数：124 | AI特征：疑似 | <div><div></div></div> | 1.0% | <p>在本文系统里，它主要为后续的CEI筛选和补盲方向判断提供参考依据。Daikon动态不变量检测框架Daikon由Ernst等人提出，</p> <p>用来从程序多次运行的运行时轨迹中推断可能成立的程序不变量（Program Invariant）。在常见的用法中，</p>                                                                                                                                                                                                                                                       |
| NO. 103 | 片段3 | 字符数：126 | AI特征：疑似 | <div><div></div></div> | 1.1% | <p>而是DL框架Fuzzing产物里“成功执行”和“崩溃触发”两组对照的数据。对同一类算子或相似的计算图结构，系统把张量形状和关键参数整理为可比较的数值键值对，</p> <p>再看哪些约束在健康样本里普遍成立、却在崩溃样本中不再成立。这些约束不能直接当作缺陷的原因，</p>                                                                                                                                                                                                                                                     |
| NO. 104 | 片段4 | 字符数：132 | AI特征：显著 | <div><div></div></div> | 1.1% |                                                                                                                                                                                                                                                                                                                                                                                                |

和直接调用LLM相比，RAG更看重给模型补充项目内的上下文。这样可以减少模型只凭通用经验作答的情况，也方便把后续新增的历史测试数据继续放入检索范围里，而不用因为数据变化就重新训练模型权重。针对DL框架故障诊断，本文主要把当前崩溃相关的测试上下文交给LLM，

NO. 105

片段5

字符数：137

AI特征：显著

1.2%

只负责把系统里已有的分析信息整理为辅助诊断报告或候选补盲测试脚本的草案。具体的集成方式会在第四章展开。代码覆盖率采集技术  
代码覆盖率用来描述测试用例触达了哪些代码区域，也是后续计算用例覆盖贡献的主要数据来源。GNU工具链里的Gcov是面向C/C++程序的覆盖率采集工具，

NO. 106

片段6

字符数：147

AI特征：疑似

1.2%

它可以对多次运行产生的覆盖率数据做合并（Merge）和差集运算，也能生成HTML格式的可视化覆盖率报告。在本课题中，系统需要在插桩编译过的PyTorch底层C++源码环境下，逐条执行测试用例并采集对应的覆盖贡献数据。借助LCOV的合并和差集功能，系统可以追踪每条用例对全局覆盖率的边际贡献。

NO. 107

片段7

字符数：125

AI特征：疑似

1.1%

进入系统的数据主要来自Fuzzing工具生成的测试脚本、执行日志等；经过解析、向量化和统计分析之后，系统向开发者提供数据管理界面和分析结果。CEI评分、故障关联分析结果、辅助诊断报告和候选补盲结果等也会在分析页面中展示。具体的功能性需求如表所示。

NO. 108

片段8

字符数：135

AI特征：显著

1.1%

测试用例数据由测试脚本、日志文件、算子拓扑结构、执行状态与覆盖数据等组成。异构存储模块由MySQL和Milvus共同承担，其中MySQL保存结构化元数据、状态字段和覆盖统计结果，Milvus保存日志向量和结构向量，两者共同实现条件查询、相似用例召回，以便后续分析。

NO. 109

片段9

字符数：125

AI特征：疑似

1.1%

故障辅助分析模块负责语义相似召回、故障辅助定位、故障关联分析和RAG辅助诊断。这些算法能力根据上层请求被调用，  
处理结果再经由服务调度层返回应用层展示。服务调度层位于应用层和算法处理层之间，  
负责接收前端请求、组织查询或分析流程，并把结果整理后返回。

NO. 110

片段10

字符数: 123

AI特征: 显著

1.0%

常规业务查询以及语义检索、RAG问答请求进入Java业务后端，  
其中语义检索和RAG问答再由Java业务后端调用Python算法后端；  
数据导入、覆盖分析、效能评估、多样性分析、故障关联分析和候选补盲生成等算法请求则直接访问Python算法后端。

NO. 111

片段11

字符数: 122

AI特征: 疑似

1.0%

故障辅助分析展示包括AI智能分析助手、故障关联分析、故障临界差分对比和用例故障定位，  
主要用于呈现相似历史用例、候选算子节点、参数差分和辅助诊断报告。数据分析与优化展示包括算子覆盖缺口分析、候选补盲生成结果、测试效能评估和测试用例多样性分布，

NO. 112

片段12

字符数: 125

AI特征: 疑似

1.1%

数据访问层使用Spring Data JPA/Hibernate作为ORM工具，  
通过实体类映射test\_case\_metadata表，  
常规查询由Repository接口派生方法完成，  
统计类的结果则使用Repository中的原生SQL聚合查询。

NO. 113

片段13

字符数: 140

AI特征: 显著

1.2%

一类是case\_uid、执行状态、覆盖贡献数据和CEI得分等结构化字段；  
另一类是由语义编码模型生成的日志向量和结构向量。结构化字段需要支持条件筛选、分页、排序和状态更新，  
因此由MySQL保存。高维语义向量需要支持相似度检索，  
因此由Milvus建立向量索引并完成相似向量召回。

NO. 114

片段14

字符数: 120

AI特征: 显著

1.0%

这种异构双库存储方式让MySQL负责元数据记录和跨库关联，  
Milvus专注于向量相似度召回，  
共同为检索、分析和展示功能提供支撑。系统详细设计与主要算法  
本章依据第三章给出的系统总体架构图展开详细设计，  
重点描述算法处理层中的核心模块，

NO. 115

片段15

字符数: 132

AI特征: 疑似



1.1%

同一条记录中可能同时包含多层运行时信息以及多线程交织产生的乱序输出。日志解析模块接收原始崩溃日志及相关测试信息，  
将其中稳定的字段提取为后续分析所需的元数据。本系统设计的LogParser模块采用分层正则匹配策略，将解析过程分解为错误分类与字段提取两个独立阶段。

NO. 116

片段16

字符数: 143

AI特征: 显著



1.2%

在字段提取阶段，模块针对每种数据类型设计了专用正则表达式族，  
具体提取的结构化字段包括以下六项：  
(1) 错误类型 (error\_type)。通过匹配Python异常类名模式 (如(\ w+Error):) 提取。对于C++层的段错误，模块通过匹配Segmentation Fault字样提取。

NO. 117

片段17

字符数: 133

AI特征: 显著



1.1%

C++底层报错包含形如at /pytorch/.../THTensorMath.cpp:41的路径信息，  
Python层报错遵循File "xxx.py", line 42格式。模块分别以两组正则提取文件名、行号与模块路径。(4) 显存信息 (memory\_info)。

NO. 118

片段18

字符数: 197

AI特征: 显著



1.7%

针对Fuzzing产物的异构特性，系统设计了两条并行的向量化路径。第一条路径为报错堆栈语义表征。模块将测试用例的error\_message字段直接输入Sentence-BERT编码器。对于执行成功的用例（无错误信息），模块构造占位文本  
"Success: [status] - Model executed OK"作为编码输入。该路径产出的向量存入Milvus的fuzzing\_logs集合，

NO. 119

片段19

字符数: 136

AI特征: 显著



1.2%

该文本经Sentence-BERT编码后产出的向量存入Milvus的fuzzing\_structure集合，  
用于多样性分析与拓扑相似性召回。两条路径共享同一个all-MiniLM-L6-v2模型实例以避免重复加载开销。每个测试用例经过该处理后产出两个独立的384维向量。

NO. 120

片段20

字符数: 141

AI特征: 显著



1.2%

保存用例状态、错误信息、分支边覆盖数、执行耗时和扩展参数等信息。Milvus侧维护fuzzing\_logs和fuzzing\_structure两个集合，  
分别对应报错堆栈语义向量和模型结构向量。两类数据不放在同一数据库中，  
主要是因为结构化筛选和高维相似度检索对存储模型的要求不同。



|                                                                                                                                                                                         |      |          |          |                        |      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------|----------|------------------------|------|
| NO. 121                                                                                                                                                                                 | 片段21 | 字符数: 122 | AI特征: 疑似 | <div><div></div></div> | 1.0% |
| <p>结构向量主键与日志向量主键同时写入parameters字段，<br/>并与结构化元数据和case_uid一同保存。后续检索时，系统可先在Milvus中召回相似向量，<br/>再依据返回的主键查询MySQL中的元数据。这种设计避免在Milvus中保存业务字段，</p>                                                |      |          |          |                        |      |
| NO. 122                                                                                                                                                                                 | 片段22 | 字符数: 150 | AI特征: 疑似 | <div><div></div></div> | 1.3% |
| <p>日志解析完成后，系统需要将结构化错误信息与测试用例的计算图结构相结合，<br/>定位出可能与崩溃相关的候选算子节点位置。本系统设计的FaultLocator模块通过逆向邻接表与DFS回溯实现该功能，<br/>帮助用户缩小排查范围。逆向邻接表的构建过程如下。系统将测试用例中的算子拓扑统一表示为有向图<math>G=(V, E)</math>，</p>      |      |          |          |                        |      |
| NO. 123                                                                                                                                                                                 | 片段23 | 字符数: 165 | AI特征: 疑似 | <div><div></div></div> | 1.4% |
| <p>故障关联分析中的候选边界条件分析模块借鉴经典的Daikon动态不变量推断范式，<br/>在成功样本与崩溃样本构成的对照组中扫描候选约束，<br/>输出候选边界条件，用于辅助分析崩溃触发条件。候选边界条件分析的输入为一个目标崩溃用例。模块先提取该用例的图拓扑结构向量，<br/>在Milvus的fuzzing_structure集合中执行余弦相似度检索，</p> |      |          |          |                        |      |
| NO. 124                                                                                                                                                                                 | 片段24 | 字符数: 136 | AI特征: 疑似 | <div><div></div></div> | 1.2% |
| <p>仅当某条候选约束同时满足“成功全票通过”与“失败全票否决”的双重验证时，<br/>该约束才被加入候选边界条件集合。上述三层扫描的双重验证判据以算法的伪代码形式给出。提取出的约束集合R经去重后按优先级排序：<br/>多变量约束排在单变量约束之前，<br/>因为多变量约束包含更丰富的变量间交互信息。</p>                             |      |          |          |                        |      |
| NO. 125                                                                                                                                                                                 | 片段25 | 字符数: 130 | AI特征: 显著 | <div><div></div></div> | 1.1% |
| <p>主要提供多样性分布观察、效能评估、算子覆盖缺口分析和候选补盲测试脚本生成。基于PCA的测试集多样性分析Fuzzing过程会持续产生测试用例。这些用例在语义向量空间中的分布，<br/>可以用于观察测试集在算子组合和参数结构上的差异。由于384维向量空间不便直接查看，</p>                                             |      |          |          |                        |      |
| NO. 126                                                                                                                                                                                 | 片段26 | 字符数: 145 | AI特征: 疑似 | <div><div></div></div> | 1.2% |



存入MySQL的parameters字段中的vis\_x与vis\_y键。前端散点图基于上述二维坐标渲染。用户可观察测试集的空间分布形态：

若散点呈现明显的簇状聚集，说明当前Fuzzing策略可能集中在有限的算子组合区域内；

若散点分布较为分散，说明测试输入在模型结构或参数组合上具有更高差异。

NO. 127

片段27

字符数：124

AI特征：显著

1.0%

因为这类算子消耗的执行资源更多，却不一定带来更高的覆盖贡献。表列出本系统的部分算子权重。系统在执行清退时，把全库CEI中位数的倍作为淘汰阈值。CEI低于这个阈值的用例在当前指标下覆盖收益相对较低，会进入级联删除流程。这一流程主要帮助识别冗余数据，

NO. 128

片段28

字符数：129

AI特征：显著

1.1%

第二步，调用Milvus的delete接口按主键删除两个集合中的对应向量。第三步，在MySQL中执行DELETE语句移除对应元数据记录。算子覆盖缺口分析

除了对单个用例做效能评估之外，系统还需要从全局视角分析测试盲区，

即在当前已出现算子和常用算子范围内，

NO. 129

片段29

字符数：137

AI特征：显著

1.2%

所有在PyTorch算子语义下合法的算子组合（排除不合理的组合，例如Softmax不能直接接Conv2d的通道输入端）。已覆盖算子对集合 $E = (i, j) \mid A[i][j] = 1$ 。盲区集合定义为：盲区集合B中的每个元素 $(o_i, o_j)$ 表示一对还没被测试的算子衔接。

NO. 130

片段30

字符数：139

AI特征：疑似

1.2%

System板限定LLM的角色边界，减少它偏离测试生成任务；

Context板块注入真实的历史数据作为参考，减少无上下文生成的问题；

Target板块指定覆盖目标，减少LLM生成已有冗余用例的可能；

Constraint板块给出语法和结构约束，让输出更便于后续检查和执行验证。

NO. 131

片段31

字符数：177

AI特征：疑似

1.5%

本系统的开发、测试和部署涉及前端、业务后端、算法后端和底层物理执行四个层面。为了让系统实现和后续实验环境保持一致，

本章和第六章采用同一套软硬件配置。表列出系统实现和实验验证所使用的主要环境配置。当前实现中，前端运行在Vite开发服务器上（默认端口5173），

通过Axios访问Java Spring Boot后端和Python FastAPI后端。

NO. 132 片段32 字符数: 121 AI特征: 显著  1.0%

Java后端（端口8080）主要负责结构化数据管理与部分代理转发，  
Python后端（端口5000）主要负责语义检索、候选边界条件分析、  
CEI分析和候选补盲建议等算法处理。底层物理执行环境部署在WSL2中的Ubuntu 22.04子系统里。

NO. 133 片段33 字符数: 131 AI特征: 疑似  1.1%

这个子系统里安装了带Gcov探针的PyTorch编译版本，  
以及独立的Python虚拟环境venv\_pytorch。物理执行层通过multiprocessing模块的spawn启动方式创建独立的子  
进程，  
在WSL环境下调用插桩后的PyTorch库执行测试脚本，

NO. 134 片段34 字符数: 171 AI特征: 显著  1.4%

该后端注册了数据导入、语义检索、统计分析、候选边界条件分析、CEI清退和补盲建议等API路由。表列出主要  
路由的方法、路径和功能摘要。语义检索接口/api/search的请求体以query字段为核心，  
用于接收自然语言查询文本。请求模型还包含top\_k、search\_mode、  
structure\_weight和log\_weight等字段，

NO. 135 片段35 字符数: 155 AI特征: 疑似  1.3%

响应体包含failed\_case、successful\_case、  
diff、batch\_context、daikon\_violations、  
daikon\_validation和boundary\_rule等字段。其中，diff用于展示参数差异，  
daikon\_validation保存待人工验证的正反样例，

NO. 136 片段36 字符数: 136 AI特征: 疑似  1.2%

/api/statistics/overview返回用例总数、崩溃率和平均执行耗时；  
/api/statistics/status返回各状态类别的用例数；  
/api/statistics/errors返回错误类型。这些结果会被封装为前端图表组件可以直接使用的JSON格式。

NO. 137 片段37 字符数: 175 AI特征: 疑似  1.5%

通过ON DUPLICATE KEY UPDATE语义在重复导入时更新已有记录。parameters字段采用JSON类型，  
保存模型结构字典、PCA降维坐标和Milvus双向量主键等扩展信息。其中Milvus双向量主键包括  
structure\_vector\_id和log\_vector\_id。Milvus侧维护两个独立的向量集合，Schema配置相同。

3.面向深度学习系统模糊测试的测试用例智能管理与分析\_第3部分

AI特征值：15.0% AI特征字符数/章节(部分)字符数：1804/12042

片段指标列表

| 序号  | 片段名称 | 字符数 | AI特征 |  |      |
|-----|------|-----|------|--|------|
| 201 | 片段1  | 153 | 显著   |  | 1.3% |
| 202 | 片段2  | 124 | 显著   |  | 1.0% |
| 203 | 片段3  | 128 | 疑似   |  | 1.1% |
| 204 | 片段4  | 142 | 显著   |  | 1.2% |
| 205 | 片段5  | 122 | 疑似   |  | 1.0% |
| 206 | 片段6  | 217 | 疑似   |  | 1.8% |
| 207 | 片段7  | 138 | 疑似   |  | 1.2% |
| 208 | 片段8  | 136 | 疑似   |  | 1.2% |
| 209 | 片段9  | 133 | 疑似   |  | 1.1% |
| 210 | 片段10 | 128 | 疑似   |  | 1.1% |
| 211 | 片段11 | 132 | 疑似   |  | 1.1% |
| 212 | 片段12 | 128 | 疑似   |  | 1.1% |
| 213 | 片段13 | 130 | 显著   |  | 1.1% |
| 214 | 片段14 | 132 | 疑似   |  | 1.1% |
| 215 | 片段15 | 127 | 疑似   |  | 1.1% |
| 216 | 片段16 | 130 | 显著   |  | 1.1% |
| 217 | 片段17 | 135 | 疑似   |  | 1.1% |
| 218 | 片段18 | 131 | 显著   |  | 1.1% |

|     |      |     |    |                        |      |
|-----|------|-----|----|------------------------|------|
| 219 | 片段19 | 142 | 疑似 | <div><div></div></div> | 1.2% |
| 220 | 片段20 | 124 | 显著 | <div><div></div></div> | 1.0% |
| 221 | 片段21 | 128 | 显著 | <div><div></div></div> | 1.1% |
| 222 | 片段22 | 129 | 疑似 | <div><div></div></div> | 1.1% |
| 223 | 片段23 | 126 | 显著 | <div><div></div></div> | 1.1% |
| 224 | 片段24 | 133 | 显著 | <div><div></div></div> | 1.1% |
| 225 | 片段25 | 123 | 疑似 | <div><div></div></div> | 1.0% |
| 226 | 片段26 | 140 | 疑似 | <div><div></div></div> | 1.2% |
| 227 | 片段27 | 133 | 显著 | <div><div></div></div> | 1.1% |
| 228 | 片段28 | 134 | 疑似 | <div><div></div></div> | 1.1% |
| 229 | 片段29 | 121 | 疑似 | <div><div></div></div> | 1.0% |
| 230 | 片段30 | 126 | 疑似 | <div><div></div></div> | 1.1% |
| 231 | 片段31 | 124 | 显著 | <div><div></div></div> | 1.0% |
| 232 | 片段32 | 148 | 疑似 | <div><div></div></div> | 1.3% |
| 233 | 片段33 | 123 | 疑似 | <div><div></div></div> | 1.0% |
| 234 | 片段34 | 125 | 疑似 | <div><div></div></div> | 1.1% |
| 235 | 片段35 | 134 | 显著 | <div><div></div></div> | 1.1% |
| 236 | 片段36 | 92  | 显著 | <div><div></div></div> | 0.8% |

片段详情

|         |     |         |         |                        |      |
|---------|-----|---------|---------|------------------------|------|
| NO. 201 | 片段1 | 字符数：153 | AI特征：显著 | <div><div></div></div> | 1.3% |
|---------|-----|---------|---------|------------------------|------|

通过卡片、表格、弹窗等形式展示测试用例、统计结果和分析信息。在实现上，前端按页面功能发起HTTP请求，其中用例管理和基础统计主要访问Java后端，数据导入、故障分析和补盲建议等功能则调用Python后端接口。FuzzHub运行总览

FuzzHub运行总览（Dashboard）是用户进入系统后的首屏，

NO. 202      片段2      字符数： 124      AI特征： 显著            1.0%

故障关联分析页面（Correlations）用于从整体上查看故障与算子的关系。页面根据后端返回的故障关联数据，展示故障用例数量、高风险算子关联和常见错误类型。中部表格按算子和算子组合汇总故障情况，便于观察问题是否集中在某些算子或特定连接关系上。

NO. 203      片段3      字符数： 128      AI特征： 疑似            1.1%

算子覆盖缺口分析页面用于查看当前测试集对算子和算子组合的覆盖情况。页面通过覆盖缺口分析接口获取统计结果，并以统计卡片、热力图和表格的形式展示。用户可以看到哪些算子已经被测试覆盖，哪些算子对在现有用例中还没有出现。这部分信息为后续补充测试提供了直接依据。

NO. 204      片段4      字符数： 142      AI特征： 显著            1.2%

测试效能评估页面用于展示CEI指标、用例效能排行和清退方案，帮助用户区分优先保留用例和候选清退用例。用户可以先预览阈值清退结果，确认后再执行处理。测试效能评估界面如图所示。测试用例多样性分布界面

多样性分析页面（Diversity）用于展示测试用例在二维PCA投影空间中的分布情况。

NO. 205      片段5      字符数： 122      AI特征： 疑似            1.0%

服务部署沿用第五章的设置：Java业务后端监听8080端口，采用Spring Data JPA和Hibernate。Python FastAPI算法后端监听5000端口。MySQL保存结构化元数据，Milvus只保存id和embedding。

NO. 206      片段6      字符数： 217      AI特征： 疑似            1.8%

物理执行实验使用multiprocessing.spawn创建独立子进程，通过p.join(timeout=10)控制单例的执行时长，超时后调用p.terminate()回收子进程，再用p.exitcode记录子进程的退出状态。实验数据集来自本系统Fuzzing工具在PyTorch框架上的测试结果。原始数据集包含2000条测试用例，每条记录包含全局唯一标识符、算子拓扑连接链（深度从1到8层不等）、输入张量形状、执行状态、

NO. 207

片段7

字符数: 138

AI特征: 疑似

1.2%

本节说明真实物理覆盖率实验所使用的执行隔离和覆盖率采集设置。物理执行隔离层运行在WSL2 Ubuntu 22.04子系统中，  
主要任务是：在带覆盖率探针的PyTorch编译版本上执行测试脚本，  
采集C++层的分支命中数据，  
通过进程级的执行隔离减少测试脚本崩溃对主服务的影响。

NO. 208

片段8

字符数: 136

AI特征: 疑似

1.2%

其中，-fprofile-arcs用于在分支跳转点插入弧计数器，  
-ftest-coverage指令让编译器为每个编译单元生成.gcno文件。链接阶段的-lgcov参数把Gcov运行时库静态链接到目标二进制中。编译完成之后，每个.cpp源文件会对应产生一个.gcno文件。

NO. 209

片段9

字符数: 133

AI特征: 疑似

1.1%

把每个弧计数器的累积值写入和.gcno同目录的.gcda文件。.gcda文件记录每条边在本次执行中的命中次数。多次执行产生的.gcda数据会累积叠加。覆盖率数据的提取通过LCOV命令行工具完成。执行流程分为三步：  
(1) 基线采集。在任何测试执行之前，运行如下命令：

NO. 210

片段10

字符数: 128

AI特征: 疑似

1.1%

在实现中，Python侧解析.info文件后提取可以用于度量覆盖贡献的数据，  
再以整数形式回写到MySQL对应的用例记录中。这一字段在CEI模型中对应E<sub>u</sub>，  
用来表示单个测试用例贡献的覆盖规模。基于multiprocessing.spawn的隔离执行模块

NO. 211

片段11

字符数: 132

AI特征: 疑似

1.1%

如果测试脚本和主Web服务共享进程空间，上述情况可能导致后端服务异常退出。为了减少这些影响，本系统采用multiprocessing模块的spawn启动方式。和默认的fork模式不同，spawn不复制父进程的内存页表，  
而是启动一个全新的Python解释器进程，

NO. 212

片段12

字符数: 128

AI特征: 疑似

1.1%

之后所有通过multiprocessing.Process创建的子进程  
都以spawn方式启动。每个测试脚本的基本执行流程如下。主进程构造Process对象，把执行函数和参数序列化后传入：  
p.join(timeout=10)让主进程阻塞等待子进程完成，

|                                                                                                                                                                                    |      |         |         |                        |      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---------|---------|------------------------|------|
| NO. 213                                                                                                                                                                            | 片段13 | 字符数：130 | AI特征：显著 | <div><div></div></div> | 1.1% |
| <p>当前批量物理执行脚本主要完成隔离执行、超时回收和探针数据注入；</p> <p>如果需要把执行状态回写到数据服务，可以采用下面的分类逻辑：</p> <p>Signal 11（段错误）和Signal 6（SIGABRT，断言失败）</p> <p>是PyTorch底层算子在非法参数组合下可能触发的两类信号。</p>                     |      |         |         |                        |      |
| NO. 214                                                                                                                                                                            | 片段14 | 字符数：132 | AI特征：疑似 | <div><div></div></div> | 1.1% |
| <p>横轴是剔除百分位（0%表示保留全部用例，100%表示剔除全部），</p> <p>左纵轴是累计边覆盖率（以全集覆盖数为基准的百分比），</p> <p>右纵轴是累计执行耗时（秒）。从曲线可以看出，</p> <p>当剔除比例从0%上升到40%时，</p> <p>覆盖率曲线只从100%缓慢降到96.8%，降幅不足4个百分点；</p>             |      |         |         |                        |      |
| NO. 215                                                                                                                                                                            | 片段15 | 字符数：127 | AI特征：疑似 | <div><div></div></div> | 1.1% |
| <p>本实验在WSL环境下用带Gcov/LCOV探针的PyTorch编译版本</p> <p>对200例子集做真实的物理执行。覆盖贡献数据来自LCOV报告中的物理命中行数，</p> <p>用来近似反映第四章公式中的<math>E_{\text{物理}}</math>。实验设置两个对照组：</p> <p>基线组按原始入库顺序依次执行全部200条用例，</p> |      |         |         |                        |      |
| NO. 216                                                                                                                                                                            | 片段16 | 字符数：130 | AI特征：显著 | <div><div></div></div> | 1.1% |
| <p>左纵轴是累计物理执行耗时（秒），右纵轴是C++层物理代码命中行数。基线组的LCOV物理命中行数终值为56857行，累计执行耗时为891.5秒；</p> <p>优选组的物理命中行数终值为56852行，累计执行耗时为595.8秒。物理命中行数只下降5行，保持率为99.991%。</p>                                   |      |         |         |                        |      |
| NO. 217                                                                                                                                                                            | 片段17 | 字符数：135 | AI特征：疑似 | <div><div></div></div> | 1.1% |
| <p>差异来自物理执行中每条用例都有固定的进程启动和探针初始化开销，</p> <p>这一开销在模拟环境中没有计入。扣除进程启动开销后，纯算子执行耗时的缩减率约为50%，</p> <p>和模拟结果的趋势一致。千例规模物理跑测</p> <p>为了在更大规模上继续观察CEI筛选的表现，</p> <p>本实验把数据集扩展到1000条用例。</p>         |      |         |         |                        |      |
| NO. 218                                                                                                                                                                            | 片段18 | 字符数：131 | AI特征：显著 | <div><div></div></div> | 1.1% |



筛选组的平均单例执行耗时（1.707秒）高于基线组（1.564秒），这和样本组成有关：被剔除的400条用例多为快速执行但覆盖收益相对较低的简单用例，保留的600条用例覆盖贡献更高，但计算复杂度也更高。崩溃用例检出数方面，筛选组为298条，基线组为312条，

NO. 219

片段19

字符数：142

AI特征：疑似

1.2%

图中结果说明，CEI筛选在当前千例数据上能够减少部分低CEI用例，并在覆盖贡献基本保持的同时降低总执行耗时。Daikon候选边界条件分析实验本节将Daikon输出作为候选边界条件分析结果，从两个方面检查它的参考价值：第一，以已知的数学约束公式为目标，检查候选约束的召回情况；

NO. 220

片段20

字符数：124

AI特征：显著

1.0%

(1) 卷积空间约束:  $input\_size + 2 * padding \geq kernel\_size$ 。(2) 转置卷积输出约束:  $output\_padding < stride$ 。(3) 分组卷积整除约束:  $in\_channels \% groups == 0$ 。

NO. 221

片段21

字符数：128

AI特征：显著

1.1%

分别构建健康组（10例成功用例）和崩溃组（10例崩溃用例），再调用Daikon模块执行不变量推断。如果模块输出的数学约束列表中包含目标公式或等价形式，就计为一次成功召回。表列出了五条目标约束的验证结果。五条已知目标约束全部被成功召回，召回率为100%。

NO. 222

片段22

字符数：129

AI特征：疑似

1.1%

如果把这类冗余约束排除掉，本组实验中没有观察到独立的错误约束。从本组实验看，Daikon模块能够召回这些已知目标公式，伴随的误报数量也处于可检查的范围。模块采用健康组全票通过、崩溃组全票否决的判据，有助于过滤偶然相关的虚假约束。按约束生成验证样例的实验

NO. 223

片段23

字符数：126

AI特征：显著

1.1%

生成验证样例时，系统先解析Daikon输出约束式涉及的参数，例如input\_height、kernel\_size、padding或groups等。然后以健康组中的最近邻成功用例作为基准样例，沿相关的参数维度执行小步长变异，分别搜索两类候选样例：

其中 $S^+$ 表示生成的正样例集合， $S^-$ 表示生成的反样例集合。这一定义把Daikon的静态不变量发现补充为可复查的样例材料：

约束不仅需要解释历史数据, 还需要提供能够被用户复查的正反参数组合。表给出三类典型约束的验证样例生成结果。每条约束都生成3个正样例和3个反样例。

样例被标记为期望成功；当系统减小输入空间尺寸或增大卷积核尺寸，让这个不等式不再成立时，样例被标记为期望触发运行时错误。这说明该约束在当前健康组和崩溃组样本中具有区分性，可以作为参数层面的排查线索，也可以为后续补充测试和验证边界条件提供参考。

本节报告模块在真实崩溃数据上给出的一个跨算子候选边界条件案例。在对一条包含Conv1d→GroupNorm算子链的崩溃用例

执行故障辅助定位和候选边界条件分析时，模块召回了10个拓扑相同的健康用例和8个崩溃用例。参数展平后，健康组和崩溃组在Conv1d的out\_channels

和GroupNorm的num\_groups两个参数上呈现出明显的分离。模块的双变量整除模板扫描检测到以下约束：这一约束在全部10个健康用例中严格成立（全票通过），在全部8个崩溃用例中都被违反（全票否决）。模块据此将该约束标记为候选边界条件，并输出到前端展示面板。

(num\_channels必须能被num\_groups整除)，但在Fuzzing场景下，Conv1d的输出通道数由上游算子的参数间接决定，当Conv1d.out\_channels由随机变异器设定为质数（如37）时，下游GroupNorm几乎不可能满足整除约束。

本文研究了PyTorch深度学习框架模糊测试结果的管理与分析问题。随着深度学习框架在实际系统中应用范围扩大，模糊测试逐渐成为发现底层算子和参数边界缺陷的重要手段。模糊测试会产生测试脚本结构、执行状态、覆盖贡献数据和崩溃日志等多类测试产物，

NO. 230

片段30

字符数: 126

AI特征: 疑似

1.1%

采用结构化元数据和语义向量协同存储的方式建立测试用例之间的关联关系。提出了基于双路向量化的相似用例召回方法，  
将测试脚本的算子结构信息和崩溃日志文本分别编码为结构向量和日志向量，  
并通过MySQL中保存的向量主键与Milvus中的向量集合完成跨库关联。

NO. 231

片段31

字符数: 124

AI特征: 显著

1.0%

在此基础上，本文实现了测试用例的统一导入、列表查询、条件筛选、  
详情查看和相似用例召回功能，  
使原本分散的测试产物可以在同一条数据流程中被组织和检索，  
为后续数据分析和故障分析提供数据基础。2. 针对用例价值分布和补充测试方向难以直观分析的问题，

NO. 232

片段32

字符数: 148

AI特征: 疑似

1.3%

结合Gcov/LCOV采集PyTorch C++层的覆盖贡献数据，  
降低单个异常用例影响主服务的风险。在真实数据集上的实验结果表明，  
在2000例模拟实验中，CEI筛选可以在覆盖率损失不足4%的情况下  
将总执行耗时缩减约60%；  
在200例物理对照实验中，物理命中行数保持率约为99.991%，

NO. 233

片段33

字符数: 123

AI特征: 疑似

1.0%

3. 针对崩溃用例中故障线索和候选边界信息不易整合分析的问题，  
实现了基于DFS拓扑回溯的故障辅助定位方法，  
对崩溃用例的算子调用链做反向遍历得到候选算子节点。在相似样本召回的基础上，  
实现了基于Daikon约束模板扫描的候选边界条件分析方法：

NO. 234

片段34

字符数: 125

AI特征: 疑似

1.1%

并在一条包含Conv1d→GroupNorm的真实崩溃用例上给出了跨算子的隐式整除候选约束。在此基础上，系统利用检索增强生成（Retrieval-Augmented Generation, RAG）方法，整理相似用例、运行信息和候选边界信息，

NO. 235

片段35

字符数: 134

AI特征: 显著

1.1%

未来的工作中，可以接入更多PyTorch版本、更多算子类型和更长时间的Fuzzing任务数据，并在大规模数据下进一步优化批量导入、增量检索和历史数据维护流程，以检验本文方法在不同运行环境下的适用性。2. 在数据分析方面，CEI模型的算子权重仍有进一步优化的空间。

NO. 236

片段36

字符数: 92

AI特征: 显著

0.8%

未来的工作中，可以扩展约束模板库以支持更多跨算子约束类型，并建立人工评分或模型评分的基准，完善候选补盲测试脚本的执行验证和回归测试流程，从而提高故障分析结果的可解释性和可复查性。

说明:

- 1、支持中、英文内容检测;
- 2、AI特征值=AI特征字符数/总字符数;
- 3、红色代表AI特征显著部分，计入AI特征字符数;
- 4、棕色代表AI特征疑似部分，未计入AI特征字符数;
- 5、检测结果仅供参考，最终判定是否存在学术不端行为时，需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。



cx.cnki.net